
Laboratório de Organização de Computadores

Lab #1 - Entrada/Saída e Estruturas Condicionais

Prof. Dr. Jean Marcos Laine
Faculdade de Computação e Informática (FCI)
Universidade Presbiteriana Mackenzie

1 Introdução e Fundamentos Teóricos

1.1 A Linguagem Assembly no Contexto de Ciência da Computação

Aprender programação Assembly e a organização de computadores é de importância fundamental para um cientista da computação. A linguagem Assembly é uma ponte direta entre o software de alto nível e a arquitetura do hardware subjacente, proporcionando um entendimento profundo do computador.

- **Entendimento da Arquitetura:** A programação Assembly permite que o aluno visualize a estrutura da CPU, os registradores, a memória e a hierarquia de armazenamento. Isso cria uma base sólida para a otimização e o desenvolvimento de software eficiente.
- **Otimização de Código:** O Assembly oferece controle granular sobre as operações da CPU. Essa habilidade é crucial para criar softwares eficientes em termos de velocidade, uso de memória e consumo de energia.
- **Depuração Avançada:** O conhecimento de Assembly possibilita uma visão detalhada do funcionamento interno dos programas, sendo extremamente útil para depurar erros complexos em níveis mais baixos do sistema.
- **Compreensão de Linguagens de Alto Nível:** Entender como o Assembly funciona ajuda a compreender como os compiladores traduzem linguagens de alto nível para instruções de máquina, ampliando a capacidade de escolher estruturas e algoritmos apropriados.
- **Segurança e Vulnerabilidades:** Ao compreender a interação entre hardware e software, o aluno pode adquirir conhecimentos essenciais em cibersegurança, identificando vulnerabilidades e escrevendo um código mais seguro.

1.2 Guia Rápido: Sintaxe e Operações Fundamentais em MIPS

O simulador MARS (MIPS Assembler and Runtime Simulator) será a nossa principal ferramenta. A sintaxe de um programa MIPS segue um padrão:

- **.data:** Seção para declaração de variáveis e dados (strings, inteiros, etc.).
- **.text:** Seção que contém o código executável do programa.
- **main:** O rótulo que indica o ponto de entrada do programa.

1.2.1 Exemplo: E/S

O exemplo abaixo demonstra como imprimir uma string e ler um valor inteiro do usuário, utilizando as ‘syscalls’ apropriadas.

```
1  .data
2  msg_prompt: .asciiz "Digite um numero: " # String para prompt
3  msg_result: .asciiz "O numero digitado foi: " # String de resultado
4
5  .text
6  .globl main
7  main:
8  # 1. Imprime a string de prompt
9  li $v0, 4          # syscall code para 'print_string'
10 la $a0, msg_prompt  # Carrega o endereco da string em $a0
11 syscall
12
13 # 2. Le um numero inteiro
14 li $v0, 5          # syscall code para 'read_int'
15 syscall            # O valor lido e retornado em $v0
16 move $s0, $v0      # Move o valor de $v0 para o registrador $s0
17
18 # 3. Imprime a string de resultado
19 li $v0, 4
20 la $a0, msg_result
21 syscall
22
23 # 4. Imprime o numero lido
24 li $v0, 1          # syscall code para 'print_int'
25 move $a0, $s0      # Carrega o valor do numero em $a0
26 syscall
27
28 # 5. Encerra o programa
29 li $v0, 10         # syscall code para 'exit'
30 syscall
31
```

Listing 1: Exemplo de I/O em MIPS

1.2.2 Exemplo: Estrutura Condicional (‘if-else’)

Em Assembly, condicionais são implementadas com instruções de desvio (‘branch’).

```
1  # Suponha que $t0 e $t1 ja contemham valores a serem comparados
2  beq $t0, $t1, if_block      # Se $t0 == $t1, desvia para 'if_block'
3  # else_block
4  # Instrucoes para o bloco else...
5  j end_if                    # Desvia para o fim do 'if' para pular o '
if_block'
6  if_block:
7  # Instrucoes para o bloco if...
8  end_if:
9  # Continuacao do programa...
10
```

Listing 2: Exemplo de Estrutura Condicional (‘if-else’)

2 Orientações Iniciais

1. Utilize o simulador MARS para escrever, montar e testar seus programas.

-
2. A montagem do programa pode ser feita com o atalho 'F3' ou pelo menu 'Run -> Assemble'.
 3. Use o modo passo-a-passo ('F7') para observar o conteúdo dos registradores e o fluxo de execução.
 4. Para a execução completa, utilize o botão 'Run' ('F5') para validar seus casos de teste.

3 Práticas de Laboratório

Exercício 1: Calculadora em MIPS (4,0 pontos)

Desenvolva um programa em linguagem Assembly MIPS que simule uma calculadora com operações básicas de soma, subtração, multiplicação e divisão com números inteiros. O programa deve ter uma estrutura de menu interativa e seguir as seguintes etapas:

1. **Apresentação e Funcionamento do Menu (1,0 pto):** O programa deve exibir um menu de opções para que o usuário escolha a operação.

===== Menu =====

1. Soma
2. Subtração
3. Multiplicação
4. Divisão
5. Sair

Solicite a escolha da operação (1/2/3/4/5).

2. **Fluxo de Controle:**

- Se a opção for 5, o programa deve encerrar.
- Se a opção for de 1 a 4, o programa deve solicitar dois números inteiros para a operação.
- Após o cálculo, o menu deve ser apresentado novamente para que o programa não se encerre.

3. **Operações e Resultados (1,0 pto):**

- Realize as operações e apresente o resultado na tela.
- Para a divisão, o programa deve mostrar o quociente e o resto quando a divisão não for exata (Exemplo: $10/3 = \text{Quociente } 3 \text{ e Resto } 1$).

4. **Tratamento de Erros (1,0 pto):** O programa deve lidar com possíveis erros de entrada, como a divisão por zero. O programa não deve permitir que o usuário digite zero como segundo operando na operação de divisão. Tratar entradas inválidas para o menu e outras situações que o grupo considerar pertinente.

Exercício 2: Comparação de Três Números (4,0 pontos)

Escreva um programa em MIPS que realize a ordenação lógica de três valores inteiros distintos fornecidos pelo usuário. O programa deve identificar o maior, o menor e o valor intermediário.

1. Interface e Aquisição de Dados (1,0 pto):

- Exibe prompts claros para a leitura de cada um dos três números inteiros.
- Armazena corretamente os valores em registradores distintos (ex: `$s0`, `$s1`, `$s2`).

2. Lógica de Comparação e Fluxo de Controle (2,0 ptos):

- **Identificação dos Extremos (1,0 pto):** Implementa corretamente as estruturas de desvio (`beq`, `bne`, `blt`, `bgt`) ou instruções de comparação (`slt`) para determinar o **maior** e o **menor** valor entre as seis combinações possíveis.
- **Identificação do Intermediário (1,0 pto):** Utiliza uma lógica consistente para isolar o valor que não é nem o máximo nem o mínimo, garantindo que o programa funcione para qualquer ordem de entrada.

3. Saída de Resultados e Organização (1,0 pto):

- **Formatação (0,5 pto):** Imprime os três resultados com labels identificadores claros (ex: "Maior valor: 15"), evitando apenas jogar o número na tela.
- **Comentários Técnicos (0,5 pto):** O código fonte deve descrever a estratégia de comparação utilizada (ex: "Comparando A com B para definir o maior provisório").

***Dica de Implementação:** Considere o uso de desvios condicionais aninhados. Desenhar um fluxograma da lógica de comparação antes de codificar pode evitar erros de branches infinitos ou resultados incorretos.*

Exercício 3: Análise de Bits e Paridade (2,0 pontos)

Desenvolva um programa em linguagem Assembly MIPS que solicite ao usuário a entrada de um único número inteiro. O programa deve realizar as seguintes análises usando operações lógicas ('AND', 'OR', 'XOR') e aritméticas, exibindo os resultados na tela com mensagens claras:

1. **Verificação de Paridade (1,0 pto):** Determine se o número inserido é par ou ímpar. Use uma operação lógica 'AND' com uma máscara de bits para verificar o bit menos significativo (LSB).
 - Se o LSB for 0, o número é par.
 - Se o LSB for 1, o número é ímpar.
2. **Status do 5º Bit (1,0 pto):** Determine o status do 5º bit do número (contando da direita para a esquerda, a partir do bit 0). O programa deve exibir "O 5º bit está ON" se o bit for 1, ou "O 5º bit está OFF" se for 0. Para isso, use uma operação lógica 'AND' com uma máscara apropriada.

4 Critérios de Avaliação e Observações

A nota da atividade é calculada com base no correto funcionamento dos programas, baseado nas pontuações que foram destacadas, e com reduções de nota para os seguintes itens indesejáveis:

ITEM INDESEJÁVEL	REDUÇÃO DE NOTA
O código não compila ou não atende aos requisitos do enunciado	ZERO
Arquivo fonte (.asm) do exercício não enviado	ZERO
Os exercícios do laboratório não foram apresentados pelo grupo	ZERO
Não entregou o relatório	-5 pts
O relatório não demonstra todos os casos de teste no exercício	-0,5 pto/exercício
Não há identificação do grupo no código-fonte e no relatório	-0,3 pto/exercício
O código não contém comentários que expliquem as etapas essenciais	-0,5 pto/exercício

5 O que entregar?

1. O relatório com as análises e tudo o que foi solicitado para a atividade, no formato PDF, usando o template disponibilizado.
2. O código fonte em MIPS (.asm), de cada exercício. Certifique-se de que os programas estejam corretos, bem organizados, comentados e legíveis.

6 Regulamento de Apresentação e Entrega dos Projetos Práticos

6.1 Apresentação e Defesa

- A apresentação do projeto é obrigatória para a atribuição de nota. Todos os integrantes devem estar presentes para a defesa do trabalho.
- Cada aluno deve demonstrar domínio sobre a implementação, sendo capaz de explicar a lógica, justificar as decisões tomadas e realizar alterações ou correções solicitadas pelo professor no momento da apresentação, se necessário.
- A ausência na apresentação ou o desconhecimento do funcionamento do código por qualquer integrante resultará em nota zero no projeto para o mesmo.

6.2 Originalidade e Autoria

- O trabalho deve ser integralmente desenvolvido pelo grupo.
- É estritamente proibida a utilização de códigos prontos, trechos de terceiros ou gerados por ferramentas de Inteligência Artificial sem o devido entendimento.
- Casos de plágio ou compartilhamento indevido de código resultarão em nota zero e poderão levar a sanções disciplinares pela coordenação.

6.3 Entrega e Prazos

- O projeto deve ser entregue exclusivamente na plataforma especificada (ex: Moodle). Envios por outros meios, como e-mail, não serão considerados válidos e a entrega não será aceita.
- O prazo de entrega deve ser rigorosamente respeitado. Entregas fora do prazo não serão aceitas.

6.4 Organização dos Grupos

- Esta atividade deve ser realizada em grupos de **2 a 4 (quatro) alunos**.
- Casos excepcionais devem ser discutidos diretamente com o professor.

Referências Bibliográficas

Hennesy, J. L.; Patterson, D. A.; Larus, J. R.; Machado Filho, N. *Organização e projeto de computadores: a interface hardware/software*. 5. ed. Rio de Janeiro: Elsevier, 2014.